Learning Deep Representations of Data Distributions

Chapter 1: An Informal Introduction

Antonio Aguirre

October 14, 2025

Journal Club (UCSC Statistics)

Today's goals

- **Problem:** high-D data live near *low-D* structure; learn it to improve **prediction**.
- **Payoff:** *prediction, denoising, completion,* and *error correction*.
- **One framework across toolkits:** analytical-PCA/PPCA, ICA/ sparse-coding / CS, Wiener/Kalman- and deep-AE, CNN ; GANs vs. diffusion—fit

$$X \xrightarrow{E} Z \xrightarrow{D} \hat{X}.$$

- **Unifying principle:** *compression* $\leftrightarrow$ *denoising* $\leftrightarrow$ *score*: denoising residual $\approx \nabla \log p$. Train to reduce coding rate with data/feature **(self-)consistency**.
- **Practice:** build algorithm-inspired networks with explicit feedback (open-ended $\rightarrow$ bi-directional $\rightarrow$ closed-loop) that scale.

Roadmap

What to Learn? (1.2)

How to Learn? (1.3)

A Unifying Approach (1.4)

Bridging Theory and Practice (1.5)

§1.1 — Intelligence, cybernetics, and the setup

- **Core claim:** the world is *predictable enough* to be compressed; intelligence = learning *low-D* structure in *high-D* data to improve prediction and action.
- **Cybernetics (closed loop):** sense $x_t \rightarrow$ estimate state $z_t \rightarrow$ choose action $u_t \rightarrow$ environment returns x_{t+1}; integrates *information*, *control*, and *decision* under feedback.
- **Modern arc:** deep learning at scale revives this loop—encoders (recognition), decoders/denoisers (generation), and policies (RL)—all trained on massive data.

What to Learn? (1.2)

Primary task (big picture)

- **Goal:** discover *low-dimensional* structure inside high-dimensional observations to enable **accurate prediction**.
- **Representation:** treat length-D windows as vectors in $\mathbb{R}^D$; the intrinsic variability lives on/near a set of dimension $d \ll D$.
- **For us:** this structure behaves like a *prior*—probability mass concentrated near a low-D set—powering inference from partial/noisy data.

Predictable & computable sequences

- **Finite-memory recursion (order d):**

$$x_{n+d} = f\big(x_{n+d-1}, \ldots, x_{n+1}, x_n\big), \qquad f \text{ computable.}$$

- **State form:** collect the last d samples

$$s_n := \begin{bmatrix} x_{n+d-1} \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^d, \qquad s_{n+1} = g(s_n).$$

- **With inputs (control):** $s_{n+1} = g(s_n, u_n),\ u_n \in \mathbb{R}^k$
- **Closed loop:** $u_n = h(s_n) \Rightarrow s_{n+1} = g\big(s_n, h(s_n)\big)$.

Learning to predict from segments

- **Data:** many windows $S_i \in \mathbb{R}^D$, each a length-D segment from the same process (with $D \gg d$).
- **Unknowns:** the minimal order d and the recursion f (or state map g).
- **Task:** recover a low-D *sufficient state* and a predictor so future values are determined from that state.

Predictability $\Rightarrow$ low-dimensional geometry

- All length-D windows produced by a predictable process lie on a d-**dimensional set** $\mathcal{S} \subset \mathbb{R}^D$.
- **Linear case:** $\mathcal{S}$ is a d-dimensional *subspace*; e.g., AR(2) $\Rightarrow$ a plane inside $\mathbb{R}^D$.
- **Nonlinear case:** $\mathcal{S}$ is a *submanifold*; identifying $\mathcal{S}$ is (under mild conditions) crucial to identifying f.
- **Multiple sources:** mixtures of processes $\Rightarrow$ mixtures of subspaces/submanifolds.

What low-dimensionality buys you (beyond prediction)

- **Completion**: with $> d$ revealed entries, infer the rest by projecting to $\mathcal{S}$.
- **Denoising**: remove small perturbations by moving x back toward $\mathcal{S}$.
- **Error correction**: fix a few arbitrary corruptions via projection + sparsity.

Probabilistic view of low-dimensionality

- **Prior (low-dim support):** $x \sim p(x)$ with mass concentrated near a d-dimensional set $\mathcal{S} \subset \mathbb{R}^D$.
- **Observation model:** $y = f(x) + n$ with known/learned forward map f and noise n.
- **Posterior:** $p(x \mid y) \propto p(y \mid x)\, p(x)$.
- **Estimators:**

$$\hat{x}_{\mathrm{MMSE}}(y) = \mathbb{E}[x \mid y], \qquad \hat{x}_{\mathrm{MAP}}(y) = \arg\max_x p(x \mid y), \qquad x \sim p(x \mid y).$$

Guiding principle

$$X \in \mathbb{R}^D \text{ concentrates near a set } \mathcal{S} \subset \mathbb{R}^D \quad \text{with} \quad \dim(\mathcal{S}) = d \ll D,$$

because the world is *predictable enough to compress*.

- **Learn** $\mathcal{S}$ (or a prior $p(x)$ concentrated near it) efficiently.
- **Use** it to *predict, denoise, complete, and correct* by moving toward $\mathcal{S}$ or high-density regions of p.

How to Learn? (1.3)

The encoder–decoder framework

$$X \xrightarrow{E} Z \xrightarrow{D} \hat{X}$$

- **Goal:** learn a low-dimensional state Z that contains the information needed to make good predictions in X-space.
- **Mechanics:** $E : \mathbb{R}^D \to \mathbb{R}^d$ (encode/infer/sufficient stats), $D : \mathbb{R}^d \to \mathbb{R}^D$ (decode/predict/denoise).
- **Training objective:**

$$\min_{E,D} \ \mathbb{E}\big[\ell(X, \hat{X})\big] \ + \ \rho(Z), \qquad \hat{X} = D(E(X)).$$

  ℓ: *consistency*, ρ: *parsimony*.
- **Bayesian read:** $E \approx$ post-estimator (e.g., $\mathbb{E}[Z \mid X]$ or MAP); $D \approx$ post-pred-estimator (e.g., $\mathbb{E}[X \mid Z]$).

Linear state–space estimation (Kalman)

$$z_{t+1} = Az_t + Bu_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, Q)$$
$$x_t = Cz_t + w_t, \qquad\qquad w_t \sim \mathcal{N}(0, R)$$

- **Enc:** $E(x_{1:t}, u_{1:t}) = \hat{z}_{t|t} = \mathbb{E}[z_t \mid x_{1:t}, u_{1:t}]$ (Kalman filter posterior mean).
- **Dec (denoise now):** $\hat{x}_{t|t} = C\,\hat{z}_{t|t}$.
- **Dec (one–step predict):** $\hat{x}_{t+1|t} = C\,(A\,\hat{z}_{t|t} + B\,u_t)$.
- **Unknown** (A, B, C)**?** Estimate model order/state via Hankel matrices and SVD (*subspace identification*), then fit (A, B, C); E, D follow.

$$x \in \mathbb{R}^D, \qquad U_k \in \mathbb{R}^{D\times k}, \qquad U_k^\top U_k = I_k$$

PCA (Principal Component Analysis):

- **Model (deterministic subspace):** $x \approx U_k z$ for some $z \in \mathbb{R}^k$ (after centering).
- **Enc/Dec:** $E(x) = U_k^\top x$ (orthogonal projection), $D(z) = U_k z \Rightarrow \hat{x} = U_k U_k^\top x$.
- **Geometry:** U_k spans the k-dimensional principal subspace (max variance directions).

$$z \sim \mathcal{N}(0, I_k), \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I_D), \quad x = U_k z + \varepsilon$$

- **Covariance:** $\mathrm{Cov}(x) = U_k U_k^\top + \sigma^2 I_D$.
- **Subspace:** $\mathrm{span}(U_k)$ equals the PCA principal subspace (up to rotation/scaling).
- **Enc/Dec:** $E(x) = U_k^\top x$, $D(z) = U_k z \Rightarrow \hat{x} = U_k U_k^\top x$.

ICA (Independent Component Analysis)

$$x = Az + \varepsilon, \quad z_1, \dots, z_k \text{ independent, non-Gaussian}, \quad A \in \mathbb{R}^{D \times k} \text{ (unknown)}$$

- **Goal:** recover independent sources z from linear mixtures x.
- **Enc/Dec:** $E(x) = W^\top x \in \mathbb{R}^k$ (learn W so $W^\top A \approx I_k$); $D(z) = Az$.
- **Practical note:** often solve via whitening ($x \mapsto \tilde{x}$) then learn an orthogonal rotation to maximize non-Gaussianity.

Sparse Coding / Dictionary Learning (DL)

$X \approx AZ, \quad X \in \mathbb{R}^{D\times N}$, $A \in \mathbb{R}^{D\times p}$ (dictionary), $Z \in \mathbb{R}^{p\times N}$ (codes), cols of Z sparse, $p \geq D$

- **Enc (per sample):** $E(x) = \arg\min_{z\in\mathbb{R}^p} \ \frac{1}{2}\|x - Az\|_2^2 + \lambda\|z\|_1$ (LASSO / ISTA or learned unrolling).
- **Dec:** $D(z) = Az$.
- **When useful:** parts-based structure, overcomplete bases, source separation.

Compressed sensing (few measurements, sparse structure)

$$y = \Phi x + \eta, \quad \Phi \in \mathbb{R}^{m \times D},\ m \ll D, \qquad x = \Psi\alpha,\ \Psi \in \mathbb{R}^{D \times p},\ \|\alpha\|_0 \le k$$

- Enc (recover sparse code):

$$\hat{\alpha} = \arg\min_{\alpha} \|\alpha\|_1 \quad \text{s.t.} \quad \|y - \Phi\Psi\alpha\|_2 \le \epsilon$$

(or LASSO: $\hat{\alpha} = \arg\min_\alpha \frac{1}{2}\|y - \Phi\Psi\alpha\|_2^2 + \lambda\|\alpha\|_1$*).*

- Dec: $\hat{x} = \Psi\hat{\alpha},\ \widehat{y} = \Phi\hat{x}$.

Nonlinear auto-encoding & denoising (Tweedie)

$$X \xrightarrow{E} Z \xrightarrow{D} \hat{X}, \qquad \min_{E,D} \mathbb{E}\big[\|X - D(E(X))\|^2\big]$$

Gaussian corruption: $Y = X + \varepsilon,\ \varepsilon \sim \mathcal{N}(0, \sigma^2 I_D),\ p_Y = p_X * \mathcal{N}(0, \sigma^2 I)$.

Tweedie (MMSE denoiser): $\mathbb{E}[X \mid Y = y] = y + \sigma^2 \nabla_y \log p_Y(y)$.

Implication: training $\hat{X} = D(E(Y))$ with squared loss makes the residual $\hat{X} - Y \approx \sigma^2 \nabla \log p_Y(Y)$, i.e., an estimate of the (smoothed) score.

Why this matters

- A plain squared–loss denoising implicitly learns the data density.
- The denoising residual estimates the score $\nabla \log p_Y$, so an encoder–decoder trained for reconstruction is also learning the geometry of $p(x)$.
- This result inspired modern score/diffusion models and justifies encode–decode pipelines as learned priors for prediction, inpainting, and generation
- Links *denoising* directly to *compression* (shorter code length, $-\log p$).

Generation via discriminative learning (GANs)

Setup and roles

- **Decoder D (generator):** $g: z \sim p(z) \mapsto \hat{X} = g(z)$ synthesizes samples.
- **Discriminator:** d learns to distinguish real X from generated $\hat{X}$.

Minimax training (discriminative view)

$$\min_{g} \max_{d} \ \ell\big(X, \hat{X} = g(z)\big)$$

- With a suitable loss, this game is equivalent to minimizing a divergence (e.g., Jensen–Shannon) between data and model.

Generation via denoising / diffusion (score-based)

Idea

- Learn iterative *denoisers* (or a score field) that move samples from noise to data.

Iterative denoising pipeline

$$z_0 \sim \mathcal{N}(0, I) \xrightarrow{g_0} z_1 \xrightarrow{g_1} \cdots \xrightarrow{g_{L-1}} z_L \approx p(X)$$

Score connection (recall Tweedie)

- Training denoisers corresponds to learning (smoothed) scores $\nabla \log p_t(x)$, which guide steps that *increase* data density.

Overview

- **Shared template:** every method learns E (low-D state) and D (explain/predict in X-space).
- **Training:** fidelity $\ell(X, \hat{X})$ (reconstruction/denoising/forecast) + parsimony $\rho(Z)$ (rank/entropy/sparsity/coding rate).

A Unifying Approach (1.4)

What is unified?

- **One lens for all methods:** learn an encode–decode pair E, D that makes data *predictable* from a low-D state Z.
- **Principle:** prefer representations that are *short* (low code length, high density) and *faithful* (good recon/prediction).
- **Scalable oracles:** (i) first-order geometry (scores/gradients, local linearization), (ii) second-order statistics (covariance/correlation).

Parsimony via compression (from $K(S)$ to coding rate)

- **Kolmogorov complexity:**

$$K(S) \;=\; \min_{p:\,U(p)=S} L(p) \quad \text{(lentgh shortest program prod. seq. } S \text{ on a univ. comp. } U\text{)}.$$

$\rightarrow$ *uncomputable*

- **Coding view:** choose an *encoder* E that maps segments $S \in \mathbb{R}^D$ to bitstrings $E(S)$, and a *decoder* D that reconstructs within precision ε.
- **Coding rate (average code length):** $R(S \mid E) \;=\; \mathbb{E}\big[L(E(S))\big] \;\approx\; \frac{1}{N}\sum_{i=1}^{N} L\big(E(S_i)\big)$.
- **Entropy link (probabilistic surrogate):** for continuous $S \sim p$, the differential entropy $h(S) = -\int p(s)\log p(s)\,ds$ quantifies typical coding length; lower entropy $\Rightarrow$ shorter codes.

Rate reduction with fidelity

- **Preference rule:** E_2 beats E_1 if it achieves the same fidelity with a larger rate drop $\Delta R = R(S|E_1) - R(S|E_2) > 0$.
- **Constrained objective (encode–decode):**

$$\min_{E,D} \; R(S \mid E) \quad \text{s.t.} \quad \operatorname{dist}\big(S,\; D(E(S))\big) \;\leq\; \varepsilon.$$

- **Lagrangian form (practical):**

$$\min_{E,D} \; \mathbb{E}[L(E(S))] \;+\; \lambda \operatorname{dist}\big(S,\; D(E(S))\big).$$

- **Link to §1.3:** with Gaussian corruption and squared loss, denoising steps move along $+\nabla \log p$ (Tweedie), $\Rightarrow$ higher density $\Rightarrow$ *shorter codes*.

Bidirectional auto-encoding (consistency)

$$X \xrightarrow{E} Z \xrightarrow{D} \hat{X}$$

- Seek compact Z that still **predicts well** in data space.
- Generic learning objective:

$$\min_{f,g} \; \big[\, \ell(X, \hat{X}) \; + \; \rho(Z) \,\big], \qquad Z = f(X), \; \hat{X} = g(Z).$$

- ℓ promotes **consistency** ($\hat{X} \approx X$); ρ promotes **parsimony/information** in Z.
- Example: classification uses a very lossy code (one-hot); continuous data generally require **lossy** coding to be realizable and sample-efficient.

Closed-loop transcription (self-consistency)

$$X \xrightarrow{E} Z \xrightarrow{D} \hat{X} \xrightarrow{E} \hat{Z}$$

- When direct $\ell(X, \hat{X})$ is impractical, enforce **self-consistency** in feature space: make Z and $\hat{Z}$ agree.
- Min–max formulation using only internal features:

$$\max_{\theta} \min_{\eta} \left[\ell(Z, \hat{Z}) + \rho(Z) \right], \quad Z = f(X; \theta),\ \hat{X} = g(Z; \eta),\ \hat{Z} = f(\hat{X}; \theta).$$

- Under mild conditions, **self-consistency** ($Z \approx \hat{Z}$) implies **consistency** ($X \approx \hat{X}$) and supports continual/incremental learning.

Deep nets as layered solvers (encode–predict–correct)

$$\text{Layer } \ell: \quad \begin{cases} \hat{X}^{(\ell)} \;=\; D_\ell\big(Z^{(\ell)}\big) & \text{decode / predict} \\ Z^{(\ell+1)} \;=\; \operatorname{prox}_{\rho_\ell}\Big(Z^{(\ell)} - \alpha_\ell\, \nabla_Z\, \ell\big(X, \hat{X}^{(\ell)}\big)\Big) & \text{encode / correct} \end{cases}$$

- **Intuition:** a deep net is a *stack of small refinement steps*—each layer decodes to the data space, measures mismatch $\ell(X, \hat{X}^{(\ell)})$, then encodes a corrected state via a simple nonlinear "prox/denoise" move.

Bridging Theory and Practice (1.5)

Bridging theory $\leftrightarrow$ practice (one slide)

$$\underbrace{X \xrightarrow{E} Z}_{\text{open-ended encode}} \quad \| \quad \underbrace{X \xrightarrow{E} Z \xrightarrow{D} \hat{X}}_{\text{bi-directional (consistency)}} \quad \| \quad \underbrace{X \xrightarrow{E} Z \xrightarrow{D} \hat{X} \xrightarrow{E} \hat{Z}}_{\text{closed loop (self-consistency)}}$$

$$\min_{f,g} \underbrace{\ell(X, \hat{X})}_{\substack{\textbf{consistency} \\ \text{data-space fidelity}}} + \underbrace{\rho(Z)}_{\substack{\textbf{parsimony / information} \\ \text{coding-rate surrogate}}} \quad \text{or} \quad \max_{\theta} \min_{\eta} \underbrace{\ell(Z, \hat{Z})}_{\substack{\textbf{self-consistency} \\ \text{feature-space agreement}}} + \rho(Z)$$

$$Z = f(X; \theta), \qquad \hat{X} = g(Z; \eta), \qquad \hat{Z} = f(\hat{X}; \theta).$$

Bridging theory $\leftrightarrow$ practice (one slide)

- ℓ **(loss)** — chooses the comparison space: data-space MSE/$-\log p(\hat{X}\,|\,X)$ for recon/generation, or feature-space alignment $\ell(Z, \hat{Z})$ when X-space metrics are ill-posed.
- ρ **(regularizer)** — enforces *low-dimensionality/information*: e.g., coding-rate proxies (entropy/variance/rank penalties), sparsity, cluster separation.
- **Why closed loop?** Enables training & test-time refinement without ground truth; supports continual learning by driving updates from *internal* mismatches $Z \leftrightarrow \hat{Z}$.
- **Scalable implementation** — unroll first-order updates; layer $\approx$ optimization step (white-box design).
- **Diagnostics (what to report):** rate reduction ΔR, fixed-point residual $\|Z - \hat{Z}\|$, and OOS prediction/denoising error.